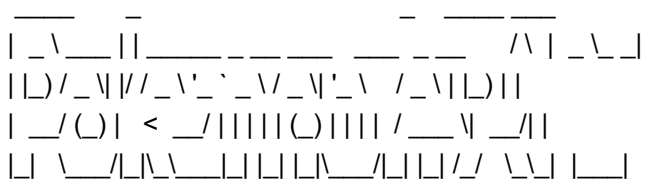
🚀 TP - 02.Bis NodeJS - Serveur API - Pokedex v2

Propriétés (5)	Valeurs
version	0.1
Bloc	Bloc2, Bloc3
classe	BTS SIO SLAM2
Compétences du référentiel	B2.1.2 - Participer à la conception de l'architecture d'une solution applicative, B2.1.5 - Identifier, développer, utiliser ou adapter des composants logiciels, B2.2.3 - Analyser et corriger un dysfonctionnement, B2.3.2 - Développer des fonctionnalités applicatives au sein d’un système de gestion de base de données (relationnel ou non)
tags	nodejs, tp, B2, api, B3



Pré requis

Ce TP est la suite de 🚀 TP - 02. NodeJS - Serveur API - Pokedex

A relire:

- 🏫 Cours - 05. API - Authentification API

Rappel des routes du projet

> ⚠ **Attention!**
>
> - Pour **rappel**, toutes les routes sont **préfixées** de: **/api/v1/**

Route	Paramètre	Nom de la route	Description
/pokemons		findAll	Renvoie TOUS les pokemons
/pokemons/random		findOneByRandom	Renvoie UN pokemon au hasard
/pokemons/{id}	**id**: integer Identifiant d'un pokemon	findOneById	Renvoie UN pokemon à partir de son **id** Si non trouvé renvoie **404**
/pokemons/{name}	**name**: string Nom d'un pokemon	findOneByName	Renvoie UN pokemon à partir de son **nom** Si non trouvé renvoie **404**

Mission 1: Limiter le nombre de requêtes à l'API

Contexte: Notre application voit de plus en plus de requêtes pointer vers elle, or, notre infrastructure est pour le moment insuffisante.

Solution proposée: Limiter le nombre de requêtes par seconde par IP

> ⚠ **Attention!**
>
> - Cette solution n'est certainement pas la seule à envisager, elle a, ici, seulement un objectif **pédagogique**

Créer une fonction qui va **vérifier si l'adresse IP** peut ou non faire la requête.

Paramètre:

- Objet **request**

  > https://expressjs.com/en/5x/api/request/

Algorithme:

1. Récupération de l'adresse IP contenue dans la requête
2. Définition d'un booléen d'autorisation de la requête
3. Récupération du moment actuel
4. Dans l'objet **Application** de l'objet **request** on ajoute un tableau de **comptage des requêtes** s'il n'existe pas

   > https://expressjs.com/en/5x/api/request/#reqapp

5. On y ajoute l'adresse IP de la requête (un **timestamp** est automatiquement ajouté)
6. Si l'adresse IP est déjà présente dans ce tableau ET que son **timestamp** est trop ancien, on le supprime
7. S'il y a plus d'entrée correspondante à cette adresse IP que la limite
   la requête est refusée
8. Sinon on ajoute le **timestamp** de la requête dans le tableau

🖊 Fonction limitRequests ⌄

```
/**
 * Fonction qui va limiter le nombre de requêtes par IP sur une période donnée
 *
 * @param {*} req
 * @param {*} maxRequests
 * @param {*} timeWindow
 * @returns boolean
 */
function limitRequests(req, maxRequests = 10, timeWindow = 60000) {
  // Récupération de l'adresse IP contenue dans la requête
  const ip = req.ip;

  // Définition d'un booléen d'autorisation de la requête
  let requestAuthorized = true;

  // Récupération du moment actuel
  const currentTime = Date.now();

  // Dans l'objet **Application** de l'objet **request** on ajoute un tableau de **comptage des requêtes**
  if (!req.app.locals.requestCounts) {
    req.app.locals.requestCounts = {};
  }

  //console.log(req.app.locals.requestCounts);

  // On y ajoute l'adresse IP de la requête (un **timestamp** est automatiquement ajouté)
  if (!req.app.locals.requestCounts[ip]) {
    req.app.locals.requestCounts[ip] = [];
  }

  // Si l'adresse IP est déjà présente dans ce tableau ET que son **timestamp** est trop ancien, on le supprime
  req.app.locals.requestCounts[ip] = req.app.locals.requestCounts[ip].filter(
    (timestamp) => currentTime - timestamp < timeWindow
  );
```

```
  // S'il y a plus d'entrée correspondante à cette adresse IP que la limite, la requête est refusée, sinon on ajoute le **timestamp** de la requête dans le tableau
  if (req.app.locals.requestCounts[ip].length >= maxRequests) {
    requestAuthorized = false;
  }else{
    // on ajoute le **timestamp** de la requête dans le tableau
    req.app.locals.requestCounts[ip].push(currentTime);
  }

  // Renvoi du jeton d'autorisation
  return requestAuthorized;
}
```

Utilisation de la fonction:

```
// 0. Limitation du nombre de requêtes par IP
if (!limitRequests(request, 1, 60000)) {
    console.log(`Too many requests from IP: ${request.ip}`);

    return response.status(429).json({ message: 'Too many requests. Please try again later.' });
}
```

☑ En savoir plus

Il existe un paquet npm qui remplit exactement cette mission
express-rate-limit

```
// npm install express-rate-limit

import rateLimit from "express-rate-limit"

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 min
  max: 100,           // per IP
  message: { error: "Too many requests" },
  standardHeaders: true,  // rate limit headers
  legacyHeaders: false
})

// apply globally
app.use(limiter)

// or per route
app.post("/login", loginLimiter, loginHandler)
```

Different limits per route

Best for: stricter limits on sensitive endpoints like login and signup

```
// strict — for auth endpoints
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5, // only 5 attempts per 15 min
  message: { error: "Too many attempts" }
})

// relaxed — for general API
const apiLimiter = rateLimit({
  windowMs: 60 * 1000,
  max: 100
})

app.post("/login",   authLimiter, ...)
app.post("/signup",  authLimiter, ...)
app.use("/api",      apiLimiter)
```

Redis-backed rate limiting - scales across servers

Best for: production apps running on multiple instances

```
// npm install rate-limiter-flexible ioredis

import { RateLimiterRedis } from "rate-limiter-flexible"
import Redis from "ioredis"

const redis = new Redis()

const limiter = new RateLimiterRedis({
  storeClient: redis,
  keyPrefix:   "ratelimit",
  points:      10,    // requests
  duration:    60,    // per minute
})

app.use(async (req, res, next) => {
  try {
    await limiter.consume(req.ip)
    next()
  } catch {
    res.status(429).json({ error: "Too many requests" })
  }
})
```

Always return the right headers

Clients need to know their limit, remaining requests, and when it resets.

```
// standard rate limit headers
RateLimit-Limit:     100
RateLimit-Remaining: 87
RateLimit-Reset:     1717884296

// when limit is hit — 429 status
res.status(429).json({
  error:   "Too many requests",
  retryAfter: 60 // seconds
})

// express-rate-limit does this
// automatically with standardHeaders: true
```